Questions

Questions::

==========
1. Application Overview
==========
- Tell me about your application

Our application is an internal auditing app used by companies to perform audit workflow in the proper manner.

Audit is basically maintaining internal information about an entity
For ex your finance department auditing will include fields like risks ,control ,documents

- Who uses it?
Senior/junior auditors ,directors ,managers

- Why do they use it?

To have an insight of what going on inside the company

==========
2. Architecture & Design
==========
- Architecture of the application

Our application follows a hybrid architecture and is currently in a transition phase
from a monolithic system to a microservices-based architecture.

At present, we have a single monolithic application that acts as the primary API Gateway.
It serves as the main entry point for client requests and handles request routing,
authentication, and some shared business logic.

Alongside the monolith, we have identified and segregated certain business functionalities
into independent microservices. These microservices are gradually being extracted from
the monolith and communicate with it through REST APIs and asynchronous messaging where required.

On the frontend, we use React with TypeScript for building the user interface, Redux for
state management, and Vite as the build tool to ensure fast development and optimized builds.

For infrastructure and supporting components:
- Redis is used for caching and performance optimization.
- Kafka is used for asynchronous event-driven communication.
- Docker is used for containerizing application components.
- Kubernetes is used for container orchestration and auto-scaling.
- AWS EC2 is used for hosting application services.
- AWS Lambda is used for specific event-driven or background tasks.
- AWS S3 is used for secure document and file storage.

- Splunk is used for centralized logging.
- Datadog is used for monitoring, metrics, and alerting.

Currently, the application uses a single shared database, which is typical during the initial phase of migrating from a monolith to microservices. This setup simplifies data consistency and reduces operational complexity while the migration is ongoing.

Overall, this architecture enables us to maintain system stability, support scalability, and modernize the application incrementally without impacting existing users.

- Why this architecture?

We chose this hybrid architecture because the application was originally built as a monolith and was already serving active users. Moving directly to a full microservices architecture would have been risky, time-consuming, and could have impacted production stability.

By keeping the monolith as the central entry point and gradually extracting selected modules into microservices, we can modernize the system incrementally without disrupting existing functionality.

This approach allows us to:
- Avoid a big-bang rewrite
- Reduce production risk
- Continue feature development during migration
- Scale only high-traffic or critical modules as microservices
- Validate microservices in production step by step

n6-

Using a single shared database during the transition simplifies data consistency and reduces operational complexity

Que on Archetecure::

1) which services converted to microservices and why?
-> We first extracted reporting, document management, notification, and background job modules into microservices. These modules were either resource-intensive, event-driven, or required independent scalability.
Core transactional audit workflows remained in the monolith to maintain data consistency and reduce migration risk during the transition phase**.**

2) how do you debug and which software do you use ?

a) **Synchronous (restTemplate)**
b) **Asynchronous(web client)**
c) **kafka**

- Advantages of the architecture

1. Gradual Migration
- Enables step-by-step transition from monolith to microservices
- Avoids risky big-bang rewrites

2. System Stability
- Existing monolith continues to serve users
- No disruption to critical business workflows

3. Independent Scalability
- High-traffic or critical modules can be scaled independently
- Kubernetes supports auto-scaling where required

4. Faster Time to Market
- New features can be developed alongside migration
- Teams are not blocked by architectural changes

5. Reduced Operational Risk
- Issues in new microservices do not impact the entire system
- Easier rollback during failures

6. Better Resource Utilization
- Microservices consume resources only when needed
- Redis and Kafka reduce unnecessary load on core services

7. Improved Observability
- Centralized logging with Splunk
- Real-time monitoring and alerts via Datadog

8. Cloud Readiness
- Docker and Kubernetes make the system cloud-native
- AWS services enable elasticity and scalability

9. Performance Optimization
- Caching with Redis reduces database load
- Event-driven processing via Kafka improves responsiveness

10. Future Scalability
- Architecture is ready for full microservices adoption
- Database separation and service isolation can be added incrementally

- Disadvantages of the architecture

1. Increased Complexity
- Managing both monolith and microservices increases architectural complexity
- Requires careful coordination between components

2. Single Database Limitation
- Shared database creates tight coupling between services
- Limits independent schema evolution
- Becomes a scalability bottleneck

3. Operational Overhead
- Requires DevOps effort for Docker, Kubernetes, monitoring, and logging
- More moving parts to manage and maintain

4. Network Latency
- Inter-service communication introduces network calls
- Slower compared to in-process monolith calls

5. Data Consistency Challenges
- Harder to manage transactions across services
- Distributed transaction handling is complex

6. Debugging Difficulty
- Issues can span multiple services
- Requires strong logging, tracing, and correlation IDs

7. Partial Microservices Benefits
- Cannot fully leverage microservices advantages due to shared database
- True service isolation is not achieved yet

8. Learning Curve
- Teams need expertise in cloud, containers, and distributed systems
- Higher onboarding time for new developers

9. Cost Overhead
- Kubernetes, monitoring tools, and cloud resources increase infrastructure cost

10. Dependency Management
- Monolith remains a critical dependency
- Failure in the monolith can still impact the entire system

==========
3. API Gateway & Traffic Management
==========

Have you used an API Gateway?

Yes, we use a dedicated API Gateway service in our microservices architecture.

Instead of the monolith acting as the entry point, we implemented a separate API Gateway layer that serves as the single entry point for all client requests.

The API Gateway is responsible for:

- Routing requests to appropriate backend microservices
- Authentication and JWT validation
- Authorization enforcement
- Centralized logging and monitoring
- Request validation
- Rate limiting and throttling
- Handling cross-cutting concerns

This separation improves:

- Scalability
- Security
- Maintainability
- Clear separation of concerns

It also allows backend services to remain lightweight and focused only on business logic.

✅ How does the API Gateway manage load?

The API Gateway acts as a centralized traffic controller.

1️⃣ Load Balancing

- It routes incoming traffic to multiple instances of backend services.
- In Kubernetes deployment, it works with Kubernetes Services and kube-proxy for load balancing.
- Only healthy pods receive traffic (health checks enabled).

2️⃣ Rate Limiting

Rate limiting restricts how many requests a client can send within a specific time window.

Purpose:

- Prevent abuse
- Avoid DDoS-style traffic spikes
- Ensure fair usage among clients

Example:

- 100 requests per minute per user

If limit exceeds → 429 Too Many Requests

3️⃣ Throttling

Throttling controls request processing speed when traffic exceeds thresholds.

Instead of rejecting immediately:

- It slows down request processing
- Protects backend services
- Maintains system stability

Throttling is especially useful during sudden traffic spikes.

4️⃣ Auto-Scaling

Auto-scaling happens at infrastructure level (Kubernetes).

- Horizontal Pod Autoscaler increases service instances
- Based on CPU, memory, or request count
- Scales down during low traffic to optimize cost

So load management is a combination of:

- API Gateway traffic control
- Kubernetes load balancing

Throttling and rate limiting are handled at the API Gateway level.

How load balancing done in your project?
Architecture Short Note – Load Balancing, Service Registry & Routing (Kubernetes Deployment)

1️⃣ Who does Load Balancing?

In Kubernetes, load balancing is handled at multiple layers:

• Kubernetes Service (ClusterIP / NodePort / LoadBalancer)
- Distributes traffic across healthy pods.
- Uses kube-proxy (iptables/IPVS) internally.
- Performs round-robin load balancing by default.

• Ingress Controller (if used)
- Handles external HTTP/HTTPS traffic.
- Routes traffic from outside cluster to internal services.

• API Gateway (Application Level)
- Routes requests to correct backend services.
- Applies rate limiting, throttling, authentication.
- Relies on Kubernetes Service for actual pod-level balancing.

So:

External LB → Ingress → API Gateway → Kubernetes Service → Pods

---

2️⃣ Do we use Service Registry?

In Kubernetes, a separate service registry like Eureka is usually NOT required.

Reason:
- **Kubernetes has built-in service discovery.**
- **Each Service gets a DNS name.**
- **Pods communicate using:**

  http://service-name:port

Kube-DNS / CoreDNS resolves service names to cluster IPs automatically.

So Kubernetes itself acts as:
- **Service Registry**
- **Service Discovery mechanism**

3️⃣ How is Routing Done?

Routing happens in layers:

A) External Routing
Client → Load Balancer → Ingress Controller

B) Gateway Routing
API Gateway inspects:
- Path (/audit/)**
- HTTP method
- Headers
- JWT claims

Then forwards to appropriate internal service.

C) Internal Routing
API Gateway calls:
http://audit-service:8080
http://auth-service:8080

Kubernetes Service forwards request to one of the healthy pods.

Final Flow:

Client
↓
Cloud Load Balancer
↓
Ingress Controller
↓
API Gateway
↓
Kubernetes Service (DNS-based discovery)
↓
Pod Instance (Load Balanced)

Key Points for Interview:

- **Kubernetes handles service discovery.**
- **kube-proxy handles pod-level load balancing.**
- **Ingress handles external routing.**
- **API Gateway handles application-level routing and traffic control.**
- **No separate Eureka needed in Kubernetes-native setup.**

==========
4. Database Design & Load Handling
==========

- How many databases do you have?

Currently, we use a single shared database across the application.

This is intentional because the system is in a transition phase from a monolithic architecture to microservices. Using a single database simplifies data consistency, reduces migration complexity, and ensures stability while services are being gradually extracted.

As part of the future roadmap, the plan is to move toward a database-per-service model once the microservices are fully isolated.

- How does your database handle load?

The database handles load through a combination of design optimizations and supporting infrastructure.

We use proper indexing on frequently queried columns to speed up read operations. Connection pooling is enabled to efficiently manage database connections and prevent connection exhaustion under high traffic.

To reduce direct database load, Redis caching is used for frequently accessed and read-heavy data. Pagination is applied for large result sets to avoid heavy queries.

Additionally, background processing and asynchronous workflows via Kafka help offload non-critical operations from synchronous database access, improving overall performance and scalability.

Connection Pooling

Connection pooling works by maintaining a pool of pre-created database connections that can be reused by multiple requests instead of creating a new connection each time.

When the application starts, the connection pool (for example, HikariCP in Spring Boot) creates a fixed number of database connections and keeps them ready.

When a request needs to access the database:
- It borrows an available connection from the pool
- Executes the query
- Returns the connection back to the pool instead of closing it

If all connections are in use, new requests wait until a connection becomes available, rather than creating unlimited connections and overwhelming the database.

This approach:
- Reduces connection creation overhead
- Improves performance under high traffic
- Prevents database connection exhaustion
- Ensures controlled and efficient use of database resources

==========
5. Caching Strategy
==========
- Is caching used in your project?
Yes, caching is used in our project.
We use Redis as a centralized in-memory cache to improve performance and reduce load on the database. Caching is mainly applied to frequently accessed, read-heavy data that does not change frequently.

- What is stored in the cache?
- When is data stored in cache?
- When and how is cache cleared?

==========
6. Logging & Monitoring
==========
- How is logging implemented?
- Which logging/monitoring tools are used?
- How is the logging tool integrated?
- How does application data reach the logging system?

==========
7. Authentication & Authorization
==========
- How is authentication managed?
- How is authorization managed?

==========
8. Inter-Service Communication
==========
- How do services communicate with each other?
- Synchronous vs asynchronous communication
- where is multithreading applied in your application ?

==========
9. Messaging / Event Streaming
==========
- Where is Kafka used in your project?
- Why is Kafka used?

==========
10. Background & Scheduled Jobs
==========
- Are there background jobs in your project?
- What kind of tasks run in the background?

==========
11. Bugs & Issue Handling
==========
- Bugs faced in the project
- How were those bugs identified?
- How were they resolved?

==========
12. Recent Contributions
==========
- Your recent contributions to the project

==========
13. Roles & Responsibilities
==========
- Your role in the team
- Your day-to-day responsibilities

==========
14. Challenges & Problem Solving
==========
- Difficulties faced in the project
- How did you tackle them?

==========
15. AOP (Aspect-Oriented Programming)
==========
- Where is AOP used in your project?
- Why was AOP used?

=====
Cloud
=====
- how do you manage load balancing ?

========
Others
========

1) **HTTP vs HTTPS**

Functional Info

We have an **internal audit application** that supports the **end-to-end audit lifecycle**.

Key modules:

- Dashboard
- Assessment area
- Project management
- Time tracking
- Audit report creation
- Recycle bin
- Audit history

Core audit objects:

- Risks and controls
- Strategic risks
- Objectives
- Workpapers

Major functionalities:

- Configurable **workflows**
- **Dynamic role- and permission-based access control**
- **Dynamic rule engine** for business logic
- **Workpaper upload and download**
- Customizable **naming and taxonomy**
- **SMTP-based email notifications**
- Automated **audit report generation**

Business user/admin flow

Admin Flow (Context Definition)

In the audit application, the **Admin** has full system-level access and configuration control.

Admin responsibilities include:

- Managing **taxonomy settings** across the application
- Defining **roles and permissions**
    - Creating new roles from existing system roles
    - Assigning permissions to roles
- **User management**
    - Creating users
    - Assigning roles to users
- Customizing the system
    - Renaming audit objects (risk, control, objective, workpaper, etc.)
    - Modifying taxonomy structures
- Configuring **SMTP settings** for email notifications

In addition to administrative capabilities, the **Admin can perform all operations available to a normal user**, including working on audits, assessments, workpapers, workflows, and reports.

User Flow (Context Definition)

A **User** has functional access based on the roles and permissions assigned to them.

User capabilities include:

- Accessing the **Assignment area**
    - Creating new assignments
    - Adding audit objects within an assignment
- Working with **Projects**
    - Linking a project to a specific audit area
    - Performing audits within that project
    - Adding predefined audit objects (risks, controls, objectives, workpapers, etc.)
- Managing audit objects
    - Creating, editing, and deleting objects
- Generating **audit reports** from assignment or project data
- Participating in **assessment workflows**
    - Being assigned to specific assessment phases
    - Receiving notifications based on assigned phases and tasks

User limitations:

- No access to **administrative areas** such as:
    - System settings
    - Statistics
    - Configuration modules

Application flow

TeamMate+ Audit – Overview

TeamMate+ Audit is a software application used by internal audit teams to **plan, execute, track, and report audits** efficiently.
It is widely used by organizations—especially in regulated industries like **banking**—to manage **risk, compliance, and governance** activities.

Who Uses TeamMate+ Audit?

1. Internal Audit Teams

- Conduct audits across departments
- Evaluate internal controls and processes
- Identify gaps, risks, and improvement areas

2. Risk Management Professionals

- Identify and assess operational, financial, and compliance risks
- Ensure risks are monitored and mitigated

3. Compliance Officers

- Ensure adherence to regulatory requirements and internal policies
- Track compliance issues and corrective actions

4. Audit Managers & Supervisors

- Plan audits and allocate resources
- Monitor audit progress and quality
- Review findings and final reports

Why Do Organizations Use TeamMate+ Audit?

Streamlined Audit Processes

- End-to-end audit lifecycle management
- Reduces manual work and spreadsheets

Enhanced Collaboration

- Centralized platform for auditors and stakeholders
- Clear visibility into tasks, findings, and status

Risk-Based Auditing

- Focuses audits on high-risk areas
- Improves early risk detection

Compliance Assurance

- Ensures audits align with regulatory standards
- Maintains audit trails and documentation

Banking Sector Example (Simple Explanation)

Q1. Who Uses TeamMate+ Audit in a Bank?

- **Internal Audit Team**
  Checks whether banking operations are safe, efficient, and compliant.
- **Risk Management Team**
  Identifies risks like fraud, credit risk, system failures, and operational issues.
- **Compliance Officers**
  Ensure adherence to RBI / central bank regulations and internal policies.

Q2. Why Do Banks Use TeamMate+ Audit?

Banks handle:

- Huge volumes of transactions
- Sensitive customer data
- Strict regulatory requirements

They must ensure:

- **Operational safety** – no fraud or transaction errors
- **Regulatory compliance** – RBI / central bank rules
- **Risk minimization** – early detection of issues
- **Accurate audit reporting** – for regulators and senior management

TeamMate+ Audit helps **automate, standardize, and control** these activities.

Q3. How Does a Bank Use TeamMate+ Audit?

1. Audit Planning

- Decide audit areas (loans, deposits, treasury, IT systems)
- Schedule audits and assign auditors
- Prioritize audits based on risk levels

2. Risk Assessment

- Identify high-risk areas:
    - Credit risk
    - Fraud risk
    - Operational risk
- Allocate more audit effort to critical areas

3. Audit Execution

- Auditors review transactions and controls
- Evidence and findings stored centrally
- Eliminates paperwork and scattered files

4. Reporting

- Automatically generates audit reports
- Clearly highlights:
    - Issues
    - Risks
    - Recommendations

5. Follow-Up (Cron-Based Tracking)

- Tracks whether audit issues are resolved
- Sends reminders until actions are closed
- Ensures accountability

Benefits to the Bank

- **Time Savings** – Less manual tracking and documentation
- **Reduced Risk** – Systematic and consistent audits
- **Improved Compliance** – Easier regulatory adherence
- **Actionable Insights** – Identifies recurring issues and trends
- **Transparency** – Clear visibility for management and regulators

---

Final Summary

✅ **In short:**
In the banking sector, **TeamMate+ Audit** is used to **manage audits, control risks, ensure regulatory compliance, and improve audit efficiency**, helping banks remain **safe, compliant, and trustworthy**.

Technical Architecture

Tech stack used

Frontend:
- React 18, TypeScript, Vite
- Redux Toolkit

Backend:
- Java 17
- Spring Boot 3.x
- Spring Security, JWT, OAuth2
- RESTful APIs, Swagger (OpenAPI)
- JPA / Hibernate

Database:
- MySQL

Messaging & Cache:
- Kafka
- Redis

DevOps & Cloud:
- Git, GitHub
- Jenkins (CI/CD pipelines)
- Docker, Kubernetes
- AWS (EC2, S3, Lambda)

Testing & Quality:
- JUnit, Mockito
- Spring Boot Test
- SonarQube

Monitoring & Logging:
- Splunk
- Datadog

Tools:
- MobaXterm
- PuTTY

Design Pattern

Adapter design pattern::

Adapted Design Pattern – Application Flow Explanation

Our application follows an **adapted Adapter-based design pattern** to bridge HTTP requests with the service layer and enforce a structured business pipeline.

Why we use this pattern

- To **decouple controllers from business logic**
- To enforce a **standard execution pipeline** for every request
- To centralize **authorization, DB handling, and post-processing**
- To make the system **extensible and maintainable**

Request Flow (End-to-End)

1. **Controller Layer**
    - Receives the HTTP request
    - Performs minimal request validation
    - Creates an instance of the **Adapter**
2. **Adapter Layer**
    - Acts as a bridge between the HTTP layer and business logic
    - Translates request data into a business-friendly format
    - Passes control to the **Execution Service**
    - Ensures the controller remains thin
3. **Execution Service (Business Pipeline)**
    - Executes a standardized pipeline:
        - User authentication & role validation

        - Permission checks
        - Connection handling
        - Prepared statement creation
        - Database operations
        - Post-operation validations (audit logs, response checks, cleanup)
4. **Specific Business Service**
    - Contains actual business logic
    - Executes domain-specific rules
    - Interacts with the database layer
5. **Database Layer**
    - Performs CRUD operations using prepared statements
    - Returns results to the service layer
6. **Post-Processing**
    - Result validation
    - Error handling
    - Audit/logging (if applicable)
    - Response preparation
7. **Response Flow**
    - Travels back through Execution Service → Adapter → Controller
    - Controller sends the final HTTP response

High-Level Flow Summary

```
Controller
    ↓
Adapter
    ↓
Execution Service (Business Pipeline)
    ↓
Specific Service
    ↓
Database
    ↑
Post-Operation Checks
    ↑
Response back to Controller
```

Key Benefits

- **Loose coupling** between layers
- **Single execution pipeline** for all business flows
- **Easy to add new services** without touching controllers
- **Cleaner controllers, reusable business logic**
- **Consistent security and DB handling**

If you want, I can:

- Convert this into **interview talking points**
- Map it to **Spring Boot components**
- Show **which GoF patterns this adapts (Adapter + Template Method)**
- Rewrite it as **architecture documentation**

Just say the word.

give an example

Alright, here's a **concrete, no-fluff example** using **Spring Boot + Java**, mapped exactly to the flow you described. This is the kind of example interviewers actually understand.

Example: Create Audit Assignment Flow

1️⃣ Controller (Thin – no business logic)

```
@RestController
@RequestMapping("/assignments")
public class AssignmentController {

    private final AssignmentAdapter assignmentAdapter;

    public AssignmentController(AssignmentAdapter assignmentAdapter)
{
        this.assignmentAdapter = assignmentAdapter;
    }

    @PostMapping
    public ResponseEntity<?> createAssignment(@RequestBody
AssignmentRequest request) {
        return assignmentAdapter.createAssignment(request);
    }
}
```

👉 Controller only:

- Accepts HTTP request
- Delegates to Adapter
- No DB, no security, no logic

2️⃣ Adapter (HTTP → Business Bridge)

```
@Component
public class AssignmentAdapter {

    private final ExecutionService executionService;
```

```
    public AssignmentAdapter(ExecutionService executionService) {
        this.executionService = executionService;
    }

    public ResponseEntity<?> createAssignment(AssignmentRequest
request) {

        AssignmentContext context = new AssignmentContext();
        context.setRequest(request);

        ExecutionResult result = executionService.execute(
                context,
                BusinessOperation.CREATE_ASSIGNMENT
        );

        return ResponseEntity.status(result.getStatus())
                             .body(result.getResponse());
    }
}
```

👉 Adapter responsibilities:

- Converts HTTP request to **business context**
- Calls execution pipeline
- Converts business response → HTTP response

3️⃣ Execution Service (Business Pipeline – Core)

This is where your **adapted design pattern** really lives.

```
@Service
public class ExecutionService {

    private final AuthService authService;
    private final BusinessServiceFactory serviceFactory;

    public ExecutionResult execute(BusinessContext context,
```

```
                                        BusinessOperation operation) {

        // 1. Authentication & Role validation
        authService.validateUser(context.getUser(), operation);

        // 2. Get business service
        BusinessService service =
                serviceFactory.getService(operation);

        // 3. Execute business logic
        Object result = service.execute(context);

        // 4. Post-operation checks
        audit(context, result);

        return ExecutionResult.success(result);
    }

    private void audit(BusinessContext context, Object result) {
        // logging / audit trail / validation
    }
}
```

👉 Pipeline enforced here:

- Security
- Service routing
- Execution
- Post-processing

4️⃣ Business Service Interface (Template-like)

```
public interface BusinessService {
    Object execute(BusinessContext context);
}
```

5️⃣ Specific Business Service (Actual Logic)

```
@Service
public class CreateAssignmentService implements BusinessService {

    private final AssignmentRepository repository;

    public CreateAssignmentService(AssignmentRepository repository)
{
        this.repository = repository;
    }

    @Override
    public Object execute(BusinessContext context) {

        AssignmentRequest req =
                ((AssignmentContext) context).getRequest();

        // Prepared statement via JPA / JDBC
        Assignment assignment = new Assignment();
        assignment.setName(req.getName());
        assignment.setCreatedBy(context.getUser().getId());

        return repository.save(assignment);
    }
}
```

👉 Only **business logic**, nothing else.

6️⃣ Factory (Routes Operation → Service)

```
@Component
public class BusinessServiceFactory {

    private final Map<BusinessOperation, BusinessService> services;

    public BusinessServiceFactory(List<BusinessService> serviceList)
{
        services = Map.of(
            BusinessOperation.CREATE_ASSIGNMENT,
            serviceList.stream()
```

```
                .filter(s -> s instanceof CreateAssignmentService)
                .findFirst()
                .orElseThrow()
        );
    }

    public BusinessService getService(BusinessOperation operation) {
        return services.get(operation);
    }
}
```

7️⃣ Flow Recap (Interview-friendly)

```
HTTP Request
    ↓
Controller (thin)
    ↓
Adapter (HTTP → Business)
    ↓
Execution Service (pipeline)
    ↓
Specific Business Service
    ↓
Database
    ↑
Post-operation checks
    ↑
HTTP Response
```

Patterns Used (Tell it straight)

- **Adapter Pattern** – Controller → Adapter → Service
- **Template Method (adapted)** – Fixed execution pipeline
- **Factory Pattern** – Resolve business service at runtime
- **Single Responsibility** – Each layer does one thing

Roles and responsibility

Roles and responsibility
==========

- Actively involved in development activities, with approximately 80% focus on new feature development and enhancements.

- Handle around 20% of sprint capacity for production support, client-reported defects, and bug fixes, ensuring minimal impact on business operations.

- Participate in technical and architectural discussions, contributing to design decisions, scalability considerations, and best practices.

- Actively participate in Agile Scrum ceremonies including:
  - Daily stand-ups
  - Sprint planning
  - Backlog grooming
  - PI increment planning

- Perform peer code reviews to ensure code quality, adherence to coding standards, performance optimization, and maintainability.

SDLC

Software Development Life Cycle (SDLC)

1. Feature discussion and high-level planning are carried out by higher management and product leadership.

2. Approved features are broken down into user stories (tickets) by the entire team, including developers and QA.

3. Steps 1 and 2 are part of PI (Program Increment) Planning, which is done for the upcoming 3 months of development and QA work.

4. Before the start of every sprint, stories are groomed, refined, estimated, and assigned to respective developers and QA engineers.

5. During sprint planning, stories are assigned to developers. Based on the number of stories (for example, 10 tickets) and available QA resources (for example, 2 QA), QA planning is done for functional and feature-wise testing.

6. Story estimation is done using story points, where:
   - 1 story point represents approximately 1 day (8 hours) of work
   - A story typically contains 2 to 3 story points

7. Once development for a story is completed, it is handed over to the QA team for functional and regression testing.

8. If QA identifies any issues, they are reported as internal bugs.

9. If a bug is related to a story developed by a specific developer, the bug is assigned back to the same developer for fixing.

10. After bug fixes, the QA team re-tests the functionality to ensure the issue is resolved and no regressions are introduced.

11. Once QA sign-off is completed, the feature is tested in the UAT environment by the Product Manager or business stakeholders.

12. As the PI approaches completion, a code freeze is applied on the staging branch.

13. After code freeze: (checked once)
   - Automation testers and developers run automated test suites
   - Developers execute unit and integration test cases
   - DevOps runs integration pipelines
   - Performance team conducts performance and load testing

14. Development activities continue on the main/UAT branch while the staging branch is used for testing and sanity validation.

15. After all validations are successful, the final build is released to production.

SDLC Flow for Client-Reported Bug

1. A bug is reported by the client and logged as a production issue or client defect in the tracking system.

2. The issue is analyzed by the team for impact, severity, and priority.

3. Based on the analysis, a dedicated team or developer is assigned to handle the bug.

4. The bug is planned and included in the next sprint under the allocated support capacity (typically 20% of the sprint).

5. The assigned developer works on the fix, following standard development practices including code changes and unit testing.

6. Once the fix is completed, the bug is moved to QA for validation and regression testing.

7. If QA finds any issues, the bug is reassigned to the developer for correction.

8. After successful QA validation, the fix is deployed to UAT for verification by the product manager or client (if required).

9. Once approved, the fix is merged into the staging branch and included in the next planned release.

10. After release, the fix is monitored in production to ensure the issue is fully resolved and no new issues are introduced.

Feature Deployment

RELEASE MANAGEMENT & DEPLOYMENT FLOW

1. FEATURE DEVELOPMENT PHASE
- Developers work on individual feature branches created from the main development branch.
- Each feature follows:
  - Code implementation
  - Unit testing
  - Local verification
  - Peer code reviews (PRs)
- All merges are done only after review approval and static checks.

2. CODE FREEZE
- Once all planned features for the release are completed, a code freeze is declared.
- During code freeze:
  - No new features are allowed.
  - Only critical bug fixes, security fixes, or release blockers are permitted.
  - Any fix requires explicit approval.
- This ensures stability and prevents scope creep before release.

3. STAGING BRANCH CREATION
- After code freeze, a staging branch is created from the stabilized development branch.
- The staging branch represents a production-like snapshot of the application.
- No direct feature development happens on this branch.

4. STAGING ENVIRONMENT DEPLOYMENT
- The staging branch is deployed to the staging environment.
- This environment closely mirrors production in terms of:
  - Infrastructure
  - Configuration
  - Data structure
  - Security settings

5. AUTOMATION & TEST EXECUTION
- Once deployed to staging, the CI/CD pipeline is triggered.
- The following automated tests are executed:
  - Unit tests
  - Integration tests
  - API contract tests
  - Regression test suites
  - Security and vulnerability scans (if applicable)
- Any test failure blocks the release and sends feedback to the team.

6. MANUAL & BUSINESS VALIDATION
- After automation passes, manual testing is performed:
  - Functional testing
  - Exploratory testing
  - User acceptance testing (UAT)

- Business stakeholders validate:
  - Feature correctness
  - Data integrity
  - End-to-end workflows

7. RELEASE BRANCH CREATION
- Once staging is fully validated, a release branch is created from the staging branch.
- This branch is strictly controlled and used only for:
  - Final bug fixes
  - Versioning
  - Release documentation
- No new features are allowed at this stage.

8. PRODUCTION DEPLOYMENT
- The release branch is deployed to the production environment via the CI/CD pipeline.
- Deployment strategies may include:
  - Blue-green deployment
  - Rolling deployment
  - Canary releases
- Health checks and smoke tests are executed post-deployment.

9. POST-DEPLOYMENT VERIFICATION
- After production deployment:
  - Application logs are monitored
  - Metrics and alerts are observed
  - Performance and error rates are validated
- Immediate rollback plans are kept ready in case of issues.

10. RELEASE CLOSURE
- Once production stability is confirmed:
  - The release branch is tagged
  - Changes are merged back to the main branch
  - Release notes are finalized
- The system is now ready for the next development cycle.

KEY BENEFITS OF THIS PROCESS:
- Controlled and predictable releases
- Early detection of defects
- Production stability
- Clear separation of development, validation, and release stages
- Reduced risk during deployment

Bugs and interview que

Bugs faced in the project

1)Latency bug

BUG SCENARIO: PERFORMANCE / LATENCY ISSUE – TIME TRACKING MODULE

ISSUE:
Users were experiencing significant latency while loading the Time Tracking screen. The page was attempting to load a large dataset (~5000 records) in a single request, resulting in slow response times and poor user experience.

INITIAL ANALYSIS:
- The first assumption was a database performance issue.
- We analyzed:
  - SQL queries
  - Execution plans
  - Index usage
  - DB response time
- Database queries were optimized and performing within acceptable limits.
- Conclusion: DB was NOT the bottleneck.

DEEPER INVESTIGATION:
- We performed end-to-end profiling of the request flow.
- Identified that the delay was occurring in the application layer.
- Root cause was found in a custom DTO mapper.

ROOT CAUSE:
- A custom mapper was using a traditional for-loop to map each entity to a DTO one-by-one.
- This resulted in:
  - High CPU usage
  - Increased object creation
  - Long processing time for large datasets
- The mapping logic was O(n) and executed for all 5000 records at once, causing noticeable latency.

SOLUTION IMPLEMENTED:
1. DATA INDEXING
   - Added proper indexing on the ID columns involved in the time tracking queries.
   - Reduced lookup and join cost during data retrieval.

2. PAGINATION IMPLEMENTATION
   - Introduced server-side pagination.
   - Instead of loading all 5000 records at once, data is now fetched in chunks (e.g., 50 or 100 records per page).
   - Significantly reduced payload size and processing time.

3. MAPPER OPTIMIZATION

- Refactored the custom mapper to avoid inefficient looping where possible.
- Optimized DTO mapping to handle smaller paginated datasets efficiently.

RESULT:
- Page load time is reduced drastically.
- CPU and memory utilization normalized.
- Improved overall user experience.
- Application became scalable for larger datasets.

KEY LEARNINGS:
- Performance issues are not always database-related.
- Application-layer processing (DTO mapping, loops, object creation) can become a major bottleneck.
- Pagination and indexing are critical for handling large datasets efficiently.

2)BUG SCENARIO: UI FIELD NOT RENDERING – DATA COMPATIBILITY ISSUE

ISSUE:
Users reported that a specific text field was not visible on the UI while loading a particular screen. The field existed in the layout configuration but was either blank or not rendered properly in the new UI.

IMPACT:
- Users were unable to view or edit a critical test-related field.
- Issue was intermittent and data-specific, making it difficult to reproduce.
- Affected only a subset of existing records.

INITIAL INVESTIGATION:
- Attempted to reproduce the issue locally and in lower environments.
- Verified:
  - UI configuration
  - Field visibility rules
  - Role-based permissions
  - API response payload
- The issue could NOT be reproduced consistently.
- New records were working fine, indicating the problem was not in the UI logic itself.

HYPOTHESIS:
- After detailed discussion and comparison between affected and non-affected records, we suspected a data-related issue rather than a UI or backend logic issue.

DATA ANALYSIS:
- Identified the database column backing the problematic text field.
- Executed queries to extract all distinct values from that column for affected records.
- While analyzing the data, we found:
  - Presence of restricted / non-printable / legacy special characters
  - Characters introduced from an older version of the UI

- These characters were not compatible with the new UI rendering engine

ROOT CAUSE:
- The old UI allowed certain unsupported characters to be persisted in the database.
- The new UI framework had stricter rendering and validation rules.
- As a result, the field failed to render when encountering those invalid characters.

SOLUTION IMPLEMENTED:
1. DATA CLEANUP SCRIPT
  - Wrote a backend SQL/data conversion script to sanitize the affected column.
  - Removed or replaced unsupported and non-renderable characters.
  - Ensured data consistency without impacting valid content.

2. PREVENTIVE VALIDATION
  - Added validation at the backend layer to prevent such characters from being saved in the future.
  - Ensured backward compatibility with the new UI.

RESULT:
- The affected field started rendering correctly for all users.
- No UI changes were required.
- Issue resolved permanently across environments.

KEY LEARNINGS:
- Legacy data can break modern UI frameworks.
- Not all UI issues are UI bugs — data integrity plays a critical role.
- Proper data validation and cleanup are essential during UI migrations.

3) Us military client facing document upload issue ,filter was changed

- How were those bugs identified?

- How were they resolved?

1. What was your contribution? -> RBAC

2. What was your biggest challenge? How you overcome that?

Problem Statement: Duplicate Survey Responses Due to Concurrent Requests

1. System Design Overview

- We have separate tables and services for:
  - Survey
  - Questions
  - Survey Assignment
  - Survey Response

- Relationships:
  - Admin creates a unique Survey.
  - A Survey contains multiple Questions.
  - The same Question can be reused in multiple Surveys.
  - A Survey can be assigned to multiple Users.
  - A User can fill multiple Surveys.

2. Survey Response Table Structure

Table: survey_response

Columns:
- id (Auto-generated Primary Key)
- user_id (Foreign Key)
- survey_id (Foreign Key)
- question_id (Foreign Key)
- response (User Answer)

3. Issue Faced

When a user was filling out a survey:

- Due to server slowness, the UI sent duplicate requests.
- The service layer had an "update or insert" logic.
- The method handling this logic was NOT synchronized.
- Two concurrent threads executed simultaneously.

Flow of failure:
Thread 1 → Checks if record exists → Not found

Thread 2 → Checks if record exists → Not found
Thread 1 → Inserts record
Thread 2 → Inserts record

Result:
Duplicate rows created in survey_response table
Same (user_id, survey_id, question_id) combination repeated

4. Why Not Use Synchronization?

- Synchronizing the method would impact performance.
- The system needed to handle high concurrency.
- Debouncing was not an option because:
  • Multiple legitimate requests were expected.
  • We did not want to block valid parallel operations.

5. Final Solution Implemented

We added a database-level UNIQUE constraint on:

(user_id, survey_id, question_id)

SQL:

```
ALTER TABLE survey_response
ADD CONSTRAINT uk_user_survey_question
UNIQUE (user_id, survey_id, question_id);
```

6. Result

- Database now prevents duplicate combinations.
- Even if two concurrent inserts happen:
  • One succeeds
  • The other fails with unique constraint violation
- Data integrity guaranteed at DB level.
- No performance degradation from application-level locking.

Conclusion:

Instead of handling concurrency at application layer,
we enforced consistency at the database layer using
a composite unique constraint.

3. To implement any functionality you faced difficulty & you need to learn some tech & how you implemented that?

Biggest Challenge:
Handling duplicate survey responses caused by concurrent API requests
in a high-concurrency system where the API was not idempotent.

System Context:
- Separate tables and services:
  - Survey
  - Questions
  - Survey Assignment
  - Survey Response

- survey_response table structure:
  id (PK)
  user_id (FK)
  survey_id (FK)
  question_id (FK)
  response

Business Rule:
Each (user_id, survey_id, question_id) combination must be unique.

Problem:
Due to server slowness, UI sent duplicate requests.
Backend had “check-then-insert” logic (update or insert).
Method was not synchronized.

Failure Flow:
  Thread 1 → Check exists → Not found
  Thread 2 → Check exists → Not found
  Thread 1 → Insert
  Thread 2 → Insert

Result:
Duplicate rows created in survey_response table.

Why Not Use Synchronization?
- Would reduce throughput.
- High concurrency system.
- Blocking threads was not acceptable.
- Frontend debouncing not reliable.

Solution:
Moved consistency enforcement to database layer.

Added composite UNIQUE constraint:

```
ALTER TABLE survey_response
ADD CONSTRAINT uk_user_survey_question
UNIQUE (user_id, survey_id, question_id);
```

Result:
- Database guarantees uniqueness.
- If two concurrent inserts occur:
  • One succeeds
  • One fails with unique constraint violation
- No application-level locking.
- No performance degradation.
- Data integrity guaranteed.

Outcome:
Issue permanently resolved.
Appreciated by team lead in retrospective.

---

4. How you optimised your API?

=> ==========
HOW I OPTIMIZED OUR APIs
==========

API optimization was not done in one place.
It was done at multiple layers:

1) Database layer
2) Caching layer
3) Application layer
4) Authorization layer
5) Infrastructure layer

1️⃣ DATABASE OPTIMIZATION

Problem:
- Slow queries in audit reports
- N+1 query issues
- Large joins across audit, risk, findings tables

What I did:

✔ Added proper indexing

- Composite indexes on (audit_id, status)
- Indexes on foreign keys
- Indexes on frequently filtered columns

✔ Solved N+1 issue
- Replaced lazy loops with JOIN FETCH / custom queries
- Used DTO projections instead of full entity load

✔ Reduced payload size
- Used pagination (Pageable)
- Limited columns using projections

✔ Optimized reporting queries
- Used native queries for heavy aggregation
- Avoided unnecessary entity hydration

Impact:
- Query time reduced significantly
- Lower DB CPU usage
- Faster report APIs

2️⃣ REDIS CACHING OPTIMIZATION

Problem:
Authorization lookup hitting DB every request.

Solution:
✔ Implemented cache-aside pattern
✔ Stored user permissions in Redis SET
✔ TTL strategy (15–30 mins)
✔ Explicit invalidation on role update

Impact:
- Reduced DB calls for auth by large margin
- Authorization became O(1) lookup

3️⃣ APPLICATION LAYER OPTIMIZATION

✔ Removed unnecessary object mapping
- Avoided deep entity graph serialization

✔ Used async processing for:
- Email notifications
- Report generation

- Background tasks

✔ Used connection pooling (HikariCP tuning)
- Optimized max pool size
- Monitored slow queries

✔ Avoided excessive logging in production
- Reduced I/O overhead

4️⃣ AUTHORIZATION OPTIMIZATION

Problem:
Heavy permission checks at service level.

Solution:
✔ Moved to AOP-based centralized permission check
✔ Avoided duplicate authorization logic in services
✔ Used Redis-based permission resolution

Impact:
- Cleaner code
- Faster permission validation
- Reduced redundant checks

5️⃣ INFRASTRUCTURE OPTIMIZATION

✔ Enabled API Gateway level rate limiting
✔ Enabled compression (GZIP)
✔ Proper thread pool configuration
✔ Monitored via logs & metrics
✔ Tuned Kubernetes resource limits

6️⃣ RESPONSE OPTIMIZATION

✔ Used pagination everywhere
✔ Avoided returning large nested JSON
✔ Used DTO instead of exposing entities
✔ Applied selective serialization

REAL PRODUCTION EXAMPLE

In reporting API:

Before:
- Multiple joins
- Full entity load
- No caching
- High response time

After:
- Optimized native query
- Added index
- Added pagination
- Moved heavy calculation async
- Cached static metadata

Result:
✔ API response time reduced
✔ DB load reduced
✔ No timeout in peak usage

SHORT INTERVIEW VERSION

"I optimized APIs at multiple layers — database indexing, query tuning, fixing N+1 issues, Redis caching for authorization, DTO projections to reduce payload size, async processing for heavy tasks, and infrastructure tuning. This significantly reduced response time and DB load while improving scalability."

==========

5. Which endpoint/API you created?
==========
DEPLOYED ROLE & PERMISSION MANAGEMENT APIs
==========

We implemented dedicated APIs to manage dynamic roles and permissions in the system. These APIs are not simple CRUD endpoints — they include internal validation, mapping logic, cache handling, and security enforcement.

1️⃣ API :: getRoleList

Endpoint:
GET /api/roles

Purpose:
Fetch all available roles (System + Custom).

Internal Logic:
- Fetch roles from database
- Identify system roles (non-editable flag)
- Mark custom roles (editable)
- Apply organization-level filtering if applicable
- Return structured role response

Response Example:

```
[
  {
    "roleId": 1,
    "roleName": "ADMIN",
    "type": "SYSTEM",
    "editable": false
  },
  {
    "roleId": 7,
    "roleName": "CUSTOM_AUDITOR",
    "type": "CUSTOM",
    "editable": true
  }
]
```

Additional Handling:
- Authorization check before returning roles
- Redis caching (optional for optimization)

---

2 API :: getPermissionListForRole

Triggered When:
User clicks on a specific role in UI.

Endpoint:
GET /api/roles/{roleId}/permissions

Purpose:
Load complete permission matrix for the selected role.

Internal Logic:

• Validate role existence
• Fetch role-permission mappings
• Group permissions by module (Survey, Audit, User Mgmt, etc.)
• Mark selected vs unselected permissions
• Handle system role restrictions (view-only if system role)

Response Example:

```
{
  "roleId": 7,
  "roleName": "CUSTOM_AUDITOR",
  "permissions": [
    {
      "module": "SURVEY",
      "permissionCode": "SURVEY_CREATE",
      "assigned": true
    },
    {
      "module": "SURVEY",
      "permissionCode": "SURVEY_DELETE",
      "assigned": false
    }
  ]
}
```

Security Handling:
• Permission to view role configuration required
• Dynamic role-permission resolution via Redis → DB fallback

3️⃣ API :: updateRolesToPermission

Triggered When:
User edits permissions and clicks SAVE.

Endpoint:
PUT /api/roles/{roleId}/permissions

Request Body:

```
{
  "permissionIds": [101, 102, 205, 309]
}
```

Purpose:
Update role-to-permission mapping.

Internal Execution Flow:

Step 1:
- Validate role exists
- Check if role is editable
- Reject update if system role

Step 2:
- Fetch existing permissions
- Calculate:
  - Permissions to add
  - Permissions to remove

Step 3:
- Perform batch insert/delete in mapping table
- Maintain transactional integrity

Step 4:
- Invalidate Redis cache:
  ROLE_PERMISSIONS:{roleId}
  USER_ROLES cache for impacted users

Step 5:
- Optional audit logging of changes

Response:

```
{
  "status": "SUCCESS",
  "message": "Role permissions updated successfully"
}
```

🚀 Why This Design Matters

✔ Clear separation between role and permission management
✔ System roles protected (non-editable)
✔ Custom roles flexible and configurable
✔ Redis cache keeps authorization fast
✔ Cache invalidation ensures immediate reflection
✔ Transaction-safe updates
✔ Fully dynamic RBAC

FLOW SUMMARY

UI → getRoleList
→ Click Role → getPermissionListForRole
→ Edit → updateRolesToPermission

→ Cache invalidation
→ Dynamic authorization reflects immediately

==========
These APIs collectively power the dynamic RBAC system
deployed in production.
==========

6. What was the DB schemas you worked on?

I primarily worked on RBAC (Role-Based Access Control) schemas and core business schemas for an enterprise audit application.

A) RBAC SCHEMAS (Most Critical Area)
---

1) users

- id (UUID, Primary Key)
- username (Unique)
- email (Unique)
- password_hash
- status
- created_at
- updated_at

Note:
UUID used because:
- Distributed microservices environment
- Avoid sequential ID exposure
- Better merge compatibility across services

2) roles

- id (UUID, Primary Key)
- role_name (Unique)
- description
- role_type (GLOBAL / ASSESSMENT)
- created_at
- updated_at

3) permissions

- id (UUID, Primary Key)

- permission_key (Unique)
- module_name
- action_type (READ / WRITE / DELETE / APPROVE)
- created_at
- updated_at

4) user_roles (Mapping Table)

- id (UUID, Primary Key)
- user_id (UUID, FK → users.id)
- role_id (UUID, FK → roles.id)
- assessment_id (UUID, Nullable, for context-based role override)
- assigned_at
- assigned_by

5) role_permissions (Mapping Table)

- id (UUID, Primary Key)
- role_id (UUID, FK → roles.id)
- permission_id (UUID, FK → permissions.id)
- created_at

Why UUID here?
- Security-sensitive module
- Prevent predictable ID enumeration
- Used across microservices
- Compatible with Redis caching keys

B) SURVEY / RESPONSE SCHEMA
---

6) survey

- id (UUID, Primary Key)
- title
- description
- created_by (UUID)
- created_at
- updated_at

7) question

- id (UUID, Primary Key)
- question_text
- question_type
- is_mandatory
- created_at

8) survey_assignment

- id (UUID, Primary Key)
- survey_id (UUID, FK)
- user_id (UUID, FK)
- assigned_at
- due_date
- status

9) survey_response

- id (UUID, Primary Key)
- user_id (UUID, FK)
- survey_id (UUID, FK)
- question_id (UUID, FK)
- response
- submitted_at

Composite Unique Constraint:
UNIQUE (user_id, survey_id, question_id)

Purpose:
Prevent duplicate entries under concurrent API calls.

C) GENERIC CORE BUSINESS SCHEMAS
---

10) audit

- id (UUID, Primary Key)
- audit_name
- audit_type
- status
- start_date
- end_date
- created_by (UUID)

11) findings

- id (UUID, Primary Key)
- audit_id (UUID, FK)
- severity
- description
- status
- owner_id (UUID)
- created_at

12) documents

- id (UUID, Primary Key)
- reference_id (UUID)
- reference_type
- s3_path
- uploaded_by (UUID)
- uploaded_at

13) notifications

- id (UUID, Primary Key)
- user_id (UUID)
- message
- is_read
- created_at

Why UUID Instead of Auto-Increment?

- Microservices architecture
- Avoid collision during data migration
- Prevent ID guessing
- Easier replication & sharding
- Safe for distributed systems

Summary (Interview Ready)

I worked extensively on:

1) RBAC schemas (users, roles, permissions, mappings)
2) Context-based authorization schemas

3) Survey & response management schemas
4) Audit core business schemas
5) Document & notification schemas

Most tables used UUID as primary key
to support distributed architecture, security, and scalability.

7. How implementing Redis & Kafka improved response time?
8. How performance testing done in your project?
9. How you know response code improved by x second?
10. SDLC of your project
11. Tell when your responsibility over feature you delivered
12. How do you know when Redis cache should flush out the data & where do you write the logic?
13. do you know multithreading and how you implemented in your project

14.not reproducible issue?

Yes — this duplicate survey response issue was not consistently reproducible.

How We Tried to Recreate It:

1) Sent multiple rapid requests from frontend.
2) Used Fiddler to fire parallel API requests.
3) Enabled Chrome DevTools network throttling (3G mode).
4) Tested with concurrent sessions.

Still not reproducible consistently.

Observation:
- It was a race condition.
- Timing dependent.
- Occurred only under specific production latency scenarios.
- Non-deterministic in local/dev environments.

Resolution Approach:
Instead of chasing reproduction,
we focused on structural prevention.

Upgraded schema and enforced:

```
UNIQUE (user_id, survey_id, question_id)
```

Now:
- Even if any client sends duplicate concurrent requests,
- Even if retries happen due to network,
- Even if new frontend is integrated,

Database prevents duplicate entries.

Key Learning:
- Some concurrency bugs are rare and timing-sensitive.
- Do not rely only on application-level fixes.
- Enforce critical business rules at database level.
- Fix the architecture, not just the symptom.

16. **Why do we need integer pk if we have uuid?**

=> UUID ensures global uniqueness and prevents ID enumeration. However, BIGINT primary keys provide better indexing, faster joins, lower storage cost, and improved write performance due to sequential inserts. So we use BIGINT internally for database efficiency and UUID externally for security and distributed system compatibility.

Authentication

Authentication in our project is implemented using JWT (JSON Web Token) and follows a stateless authentication model.

The application is stateless, and authentication is handled at the API Gateway level. Spring Security is used to validate and authenticate requests using a chain of security filters.

When a user logs in:
- If no JWT is present, the API Gateway validates the user credentials (user ID and password) against the database.
- Once the credentials are successfully verified, the system generates a JWT.

The generated JWT contains:
- User ID
- User roles
- Token metadata (expiry, issuer, etc.)

Two types of tokens are issued:
1. Access Token
2. Refresh Token

The Access Token:
- Is short-lived
- Is stored in the frontend Redux store
- Is sent with every API request in the Authorization header
- Is used to authenticate and authorize the user for each request

The Refresh Token:
- Is long-lived
- Is stored securely in browser cookies
- Is used only when the access token expires or becomes invalid

For every incoming request:
- The API Gateway validates the access token using Spring Security filters
- If the token is missing, expired, the request is rejected

If the access token is expired:
- The refresh token is sent to generate a new access token
- If the refresh token is also invalid, the user is logged out and access is denied

Overall, authentication is enforced through a combination of JWT-based stateless security and Spring Security filter chains.

Authorization

==========
UPDATED AUTHORIZATION FLOW (Dynamic Role & Permission Lookup)
==========

Systems roles :: system roles has some non editable and viewable permissions to create custom permission we need to copy system roles and then edit permission inside them

5 System roles :: Admin, reviewer, observer , auditor, business contact

Authorization in our project follows a role-to-permission based access control model, but roles and permissions are resolved dynamically on every request.

Each role in the system is mapped to a defined set of permissions for specific functional areas of the application. These permissions control access at both API and business logic levels.

🔐 Authentication Phase (JWT Based)

When a user is authenticated:

- A JWT is generated containing only user identity (userId / username)
- Roles and permissions are NOT permanently trusted from the token
- The JWT is used only to prove identity and validate authenticity.

🚀 Request Processing & Authorization Phase

For every incoming request:

1️⃣ JWT is validated
   - Signature verified using secret key
   - Expiration checked

2️⃣ User identity is extracted from JWT (userId / username)

3️⃣ Roles and permissions are fetched dynamically:
   - First lookup in Redis (fast in-memory cache)
   - If not present, fetch from database
   - Cache the result in Redis

4️⃣ Authorization checks are performed:
   - User’s roles are mapped to permissions
   - Requested resource is matched against required permission
   - Access is granted or denied

This ensures:
✔ Immediate reflection of role or permission updates
✔ Centralized and consistent authorization
✔ No stale role data from old tokens

🧩 AOP-Based Authorization Enforcement

To enforce authorization consistently:

- Custom annotations are defined on controller or service methods
- An authorization aspect intercepts method execution
- The aspect retrieves the authenticated user identity
- It fetches current roles and permissions (from Redis/DB)
- It validates required permissions before allowing execution

If required permission is missing:

- Execution is blocked
- Authorization exception is thrown
- Business logic is not executed

⚡ Performance Optimization Using Redis

- Role-to-permission mappings are cached in Redis
- Redis is used for fast lookup on every request
- Cache is refreshed or invalidated when role/permission updates occur
- Database remains the source of truth

This provides:
✔ Fast authorization checks
✔ Immediate permission updates
✔ Scalable performance

🛡️ SecurityContext in This Architecture

SecurityContext is a Spring Security component that stores security-related information for the current request.

In our updated flow:

After JWT validation:
- An Authentication object is created using user identity

- This object is stored in SecurityContext
- Roles/permissions are resolved dynamically during authorization

SecurityContext contains:
- Authenticated user identity
- Authentication status
- Security metadata

Roles and permissions are not blindly trusted from JWT;
they are validated dynamically before authorization checks.

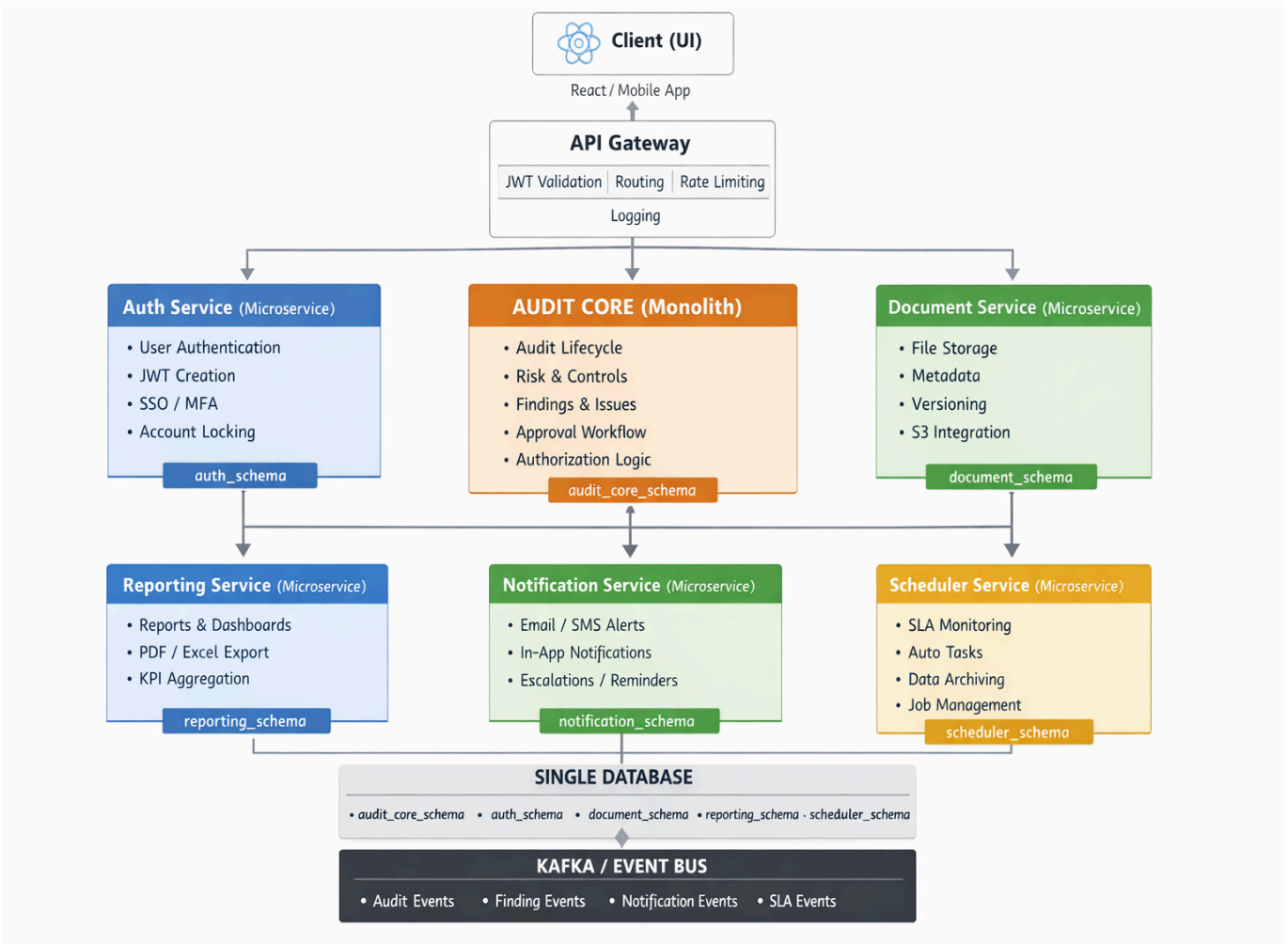
Spring Security automatically:
- Creates SecurityContext at request start
- Populates it after authentication
- Clears it after request completion

FINAL SUMMARY

JWT → Used only for identity verification
Redis → Stores latest role-permission mappings
Database → Source of truth
AOP → Enforces permission checks
SecurityContext → Holds authenticated user for current request

This approach ensures centralized, dynamic, secure, and scalable authorization
with immediate reflection of role changes.
==========

Diagram


Client (UI)
React / Mobile App
API Gateway
JWT Validation
Routing
Rate Limiting
Logging
Auth Service (Microservice)
• User Authentication
• JWT Creation
• SSO / MFA
• Account Locking
auth_schema
AUDIT CORE (Monolith)
• Audit Lifecycle
• Risk & Controls
• Findings & Issues
• Approval Workflow
• Authorization Logic
audit_core_schema
Document Service (Microservice)
• File Storage
• Metadata
• Versioning
• S3 Integration
document_schema
Reporting Service (Microservice)
• Reports & Dashboards
• PDF / Excel Export
• KPI Aggregation
reporting_schema
Notification Service (Microservice)
• Email / SMS Alerts
• In-App Notifications
• Escalations / Reminders
notification_schema
Scheduler Service (Microservice)
• SLA Monitoring
• Auto Tasks
• Data Archiving
• Job Management
scheduler_schema
SINGLE DATABASE
• audit_core_schema • auth_schema • document_schema • reporting_schema • scheduler_schema
KAFKA / EVENT BUS
• Audit Events • Finding Events • Notification Events • SLA Events

API created

@Async

==========
@Async — History Service Summary
==========

1) Purpose in History Service

We use @Async in History Service to:
- Decouple audit/history persistence from main business flow
- Reduce API response time
- Prevent history failure from breaking main request
- Achieve eventual consistency

History is treated as non-blocking, secondary operation.

2) Basic Implementation

```
@EnableAsync   // required at configuration level

@Service
public class HistoryService {

    @Async("historyExecutor")
    public CompletableFuture<Void> saveAsync(HistoryEvent event) {
        try {
            historyRepository.save(event);
        } catch (Exception ex) {
            log.error("History persistence failed for {}", event.getId(), ex);
        }
        return CompletableFuture.completedFuture(null);
    }
}
```

3) How It Works Internally

Caller Thread
↓
Spring AOP Proxy
↓
TaskExecutor.submit()
↓
Separate background thread executes saveAsync()

Key Points:
- Runs in different thread
- Separate transaction context

- Exceptions do NOT propagate to caller
- Main request returns immediately

4) Ensuring Main Request Does Not Fail

- Async method runs in separate thread
- Exceptions are caught inside method
- No exception bubbles back to controller
- History failure does not affect API response

5) Critical Data Consistency Requirement

History must run ONLY after main transaction commits.

Correct pattern:

```
@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
public void handle(TaskUpdatedEvent event) {
    historyService.saveAsync(event);
}
```

This ensures:
- No history written for rolled-back transactions

6) Retry for Transient Failures

```
@Async("historyExecutor")
@Retryable(
    value = { Exception.class },
    maxAttempts = 3,
    backoff = @Backoff(delay = 2000)
)
public CompletableFuture<Void> saveAsync(HistoryEvent event) {
    historyRepository.save(event);
    return CompletableFuture.completedFuture(null);
}
```

Solves:
- Temporary DB failure
- Connection pool hiccups
- Deadlocks

7) Thread Pool Configuration

```
@Bean(name = "historyExecutor")
public Executor historyExecutor() {
    ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
    executor.setCorePoolSize(5);
    executor.setMaxPoolSize(10);
    executor.setQueueCapacity(500);
    executor.initialize();
    return executor;
}
```

Prevents:
- Blocking request threads
- Silent task dropping
- Thread exhaustion

8) Limitations of @Async for History

- In-memory execution (lost if app crashes)
- No durability guarantee
- No built-in retry unless added
- No delivery tracking
- No exactly-once guarantee

9) When @Async Is Acceptable for History

- Audit is non-critical
- Minor data loss acceptable
- Low-to-moderate traffic
- Eventual consistency allowed

10) When @Async Is NOT Enough

If history is:
- Compliance-critical
- Financially regulated
- Requires guaranteed persistence
- Needs high throughput

Better approach:
- Outbox Pattern
- Kafka-based event-driven architecture

11) Interview-Ready Summary

In our system, History Service was executed asynchronously to prevent audit logging from increasing API latency. The async method runs in a dedicated thread pool and is triggered only after the main transaction commits using @TransactionalEventListener(AFTER_COMMIT). Failures are logged and retried when necessary. Since @Async is thread-based and not crash-safe, stronger durability guarantees would require an event-driven approach such as Kafka or the Outbox pattern.

==========
END
==========

StatusCodeExceptionHandling

==========
Exception Handling Using @ResponseStatus
(Handling Specific HTTP Status Codes)
==========

1) Purpose

@ResponseStatus is used to map a specific exception to a specific HTTP status code.

It allows:
- Clean separation of business logic and HTTP response logic
- Automatic HTTP status mapping
- Avoiding manual ResponseEntity creation everywhere

2) Basic Usage on Custom Exception

```
@ResponseStatus(HttpStatus.NOT_FOUND)
public class HistoryNotFoundException extends RuntimeException {
    public HistoryNotFoundException(String message) {
        super(message);
    }
}
```

When this exception is thrown:

```
throw new HistoryNotFoundException("History not found");
```

Spring automatically returns:

HTTP 404 Not Found

3) With Custom Reason

```
@ResponseStatus(
    value = HttpStatus.BAD_REQUEST,
    reason = "Invalid history request"
)
public class InvalidHistoryRequestException extends RuntimeException {
}
```

Response:
HTTP 400 Bad Request
Body:
"Invalid history request"

Note:
Using "reason" produces default error response body.

4) Using @ExceptionHandler (More Control)

For structured JSON responses, prefer global exception handling:

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(HistoryNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public ErrorResponse handleHistoryNotFound(
            HistoryNotFoundException ex) {

        return new ErrorResponse(
            404,
            ex.getMessage(),
            LocalDateTime.now()
        );
    }
}
```

This gives:
- Custom JSON body
- Control over structure
- Consistent API contract

5) Handling Multiple Status Codes

Different exceptions → different status codes:

```
@ResponseStatus(HttpStatus.NOT_FOUND)
public class HistoryNotFoundException extends RuntimeException {}

@ResponseStatus(HttpStatus.BAD_REQUEST)
public class InvalidHistoryRequestException extends RuntimeException {}

@ResponseStatus(HttpStatus.CONFLICT)
public class DuplicateHistoryException extends RuntimeException {}
```

6) When Exception Is Thrown in Service

```
@Service
public class HistoryService {

    public History getHistory(Long id) {
        return historyRepository.findById(id)
            .orElseThrow(() ->
                new HistoryNotFoundException("History not found"));
    }
}
```

Spring automatically converts exception → HTTP status.

7) Difference: @ResponseStatus vs ResponseEntity

@ResponseStatus:
- Static mapping
- Cleaner
- Less boilerplate
- Good for simple APIs

ResponseEntity:
- Dynamic status
- Full response control
- Used when status depends on runtime condition

8) Limitations of @ResponseStatus

- Fixed HTTP status
- Harder to return dynamic error body
- Not ideal for complex error responses
- Does not handle validation errors automatically

9) Best Practice in Production

For structured APIs:

Use:
- @RestControllerAdvice
- @ExceptionHandler
- Custom error response object

Avoid using "reason" in @ResponseStatus for REST APIs

because it limits response body customization.

10) Interview-Ready Summary

@ResponseStatus allows mapping custom exceptions to specific HTTP status codes declaratively. When the exception is thrown, Spring automatically converts it to the defined HTTP status. For more structured error handling and custom JSON responses, it is recommended to use @RestControllerAdvice with @ExceptionHandler instead of relying solely on @ResponseStatus.

==========
END
==========

==========
How Exceptions Are Thrown & Detected in Spring Boot
==========

1) Do We Need to Explicitly Throw Exceptions?

Yes — for custom business exceptions, you must explicitly throw them.

Example:

```
if (history == null) {
    throw new HistoryNotFoundException("History not found");
}
```

Spring does NOT automatically throw your custom exceptions.
You define when business logic is violated.

2) Where Can Exceptions Be Thrown?

Exceptions can originate from:

• Controller layer
• Service layer
• Repository layer
• Validation layer
• Framework itself (Spring, Hibernate, Jackson, etc.)

Spring detects any unhandled exception that propagates up to the DispatcherServlet.

3) How Spring Detects Exceptions

Flow:

Controller → Service → Repository
↓
Exception thrown
↓
Bubbles up call stack
↓
Spring DispatcherServlet catches it
↓
Checks:
- @ExceptionHandler
- @RestControllerAdvice
- @ResponseStatus
- Default Exception Resolver
↓
Converts to HTTP response

4) Example: Service Throws Exception

```
@Service
public class HistoryService {

    public History getHistory(Long id) {
        return historyRepository.findById(id)
            .orElseThrow(() ->
                new HistoryNotFoundException("History not found"));
    }
}
```

If not caught manually,
Spring automatically routes it to exception handlers.

5) Framework-Thrown Exceptions (Automatic)

Some exceptions are thrown automatically by Spring:

• MethodArgumentNotValidException (validation failure)
• HttpMessageNotReadableException (invalid JSON)
• DataIntegrityViolationException (DB constraint violation)
• MissingServletRequestParameterException

You do NOT throw these manually.
Spring throws them internally.

6) Can Spring Detect Exceptions From Anywhere?

Spring only handles exceptions that:

• Occur inside Spring-managed beans
• Happen during HTTP request processing
• Propagate to the DispatcherServlet

It will NOT detect:
• Exceptions caught and swallowed manually
• Exceptions inside separate threads unless handled properly
• Exceptions outside request lifecycle

7) What Happens If You Catch It?

```
try {
    historyRepository.save(entity);
} catch (Exception e) {
    log.error("Failed");
}
```

If you catch and do NOT rethrow,
Spring never sees it.

To let Spring handle it:

```
catch (Exception e) {
    throw new HistorySaveException("Failed to save history");
}
```

8) Async Case (Important)

For @Async methods:

• Exception does NOT propagate to controller thread.
• Must handle inside async method.
• Or use CompletableFuture and handle via exceptionally().

9) Summary

• Business exceptions must be explicitly thrown.
• Spring automatically catches uncaught exceptions.
• Exceptions bubble up the call stack.

• @ExceptionHandler / @ResponseStatus decide HTTP response.
• If you catch and suppress exception, Spring cannot detect it.
• Async exceptions do not propagate to request thread.

==========
END
==========

Deployment Strategy

==========
Rolling Update Deployment Strategy
==========

1) Purpose

Rolling Update is a deployment strategy where:
- New version instances are deployed gradually
- Old version instances are terminated progressively
- System remains available during deployment
- No full downtime occurs

Used in:
- Microservices
- Kubernetes-based deployments
- High-availability systems

2) How It Works

Instead of replacing all instances at once:

Old Version (v1):
Instance 1
Instance 2
Instance 3
Instance 4

Step-by-step process:

Step 1:
Terminate Instance 1 (v1)
Start Instance 1 (v2)

Step 2:
Terminate Instance 2 (v1)
Start Instance 2 (v2)

Continue until all are upgraded.

Traffic always flows to healthy instances.

3) Example in Kubernetes

```
spec:
  replicas: 4
```

```
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1
      maxSurge: 1
```

Meaning:
- At most 1 pod can be unavailable
- At most 1 extra pod can be created temporarily

4) Key Parameters

• maxUnavailable
  Number of old pods that can go down during update.

• maxSurge
  Extra pods allowed above desired replica count.

• Readiness Probe
  Ensures traffic only goes to healthy pods.

• Liveness Probe
  Restarts unhealthy pods.

5) Traffic Flow During Deployment

Load Balancer
↓
Healthy Pods Only
↓
Old + New versions coexist temporarily

Once new pod is Ready:
- It starts receiving traffic
- Old pod is removed

6) Advantages

• Zero downtime
• No traffic interruption
• Safe gradual rollout
• No duplicate environments required
• Resource-efficient

7) Limitations

• Old and new versions coexist → must be backward compatible
• DB schema changes must be handled carefully
• Slower rollout compared to recreate strategy

8) Data Consistency Considerations

When doing rolling update:

✔ Use backward-compatible APIs
✔ Avoid breaking DB changes
✔ Use feature flags if needed
✔ Apply DB migrations before deployment

Bad Example:
- New code expects new DB column
- Old pods still running → may break

9) Comparison With Other Strategies

Rolling Update:
- Gradual replacement
- No downtime
- Mixed versions temporarily

Blue-Green:
- Two separate environments
- Instant switch
- Higher infrastructure cost

Canary:
- Small % traffic to new version
- Risk-controlled rollout

10) Failure Handling

If new pod fails readiness:

- Kubernetes stops rollout
- Old pods continue serving traffic
- Rollback can be triggered

Deployment does NOT take down entire system.

11) When To Use

✔ Stateless services
✔ High-availability APIs
✔ Microservices
✔ Kubernetes environments

Avoid if:
✖ Breaking schema changes
✖ Major incompatible changes

12) Interview-Ready Summary

Rolling Update is a zero-downtime deployment strategy where instances are gradually replaced with newer versions while keeping the service available. Traffic is routed only to healthy pods using readiness checks. It requires backward compatibility since old and new versions coexist temporarily. Kubernetes manages rollout using parameters like maxUnavailable and maxSurge, ensuring controlled and safe deployment.

==========
END
==========

Response code

==========
Important HTTP Response Codes & Meaning
==========

1) Purpose

HTTP response codes indicate:
- Whether request succeeded
- Whether client made an error
- Whether server failed
- What action client should take next

Format:
HTTP Status Code = 3-digit number

2) Status Code Categories

1xx → Informational
2xx → Success
3xx → Redirection
4xx → Client Error
5xx → Server Error

==========
2xx — Success Codes
==========

200 OK

Request succeeded.
Most common success response.

Used for:
✔ GET successful
✔ PUT successful
✔ General success

201 Created

Resource successfully created.

Used for:
✔ POST that creates new resource

Example:
POST /history → returns 201

204 No Content

Request successful but no response body.

Used for:
✔ DELETE
✔ Successful update with no return payload

==========
3xx — Redirection Codes
==========

301 Moved Permanently

Resource permanently moved.

302 Found

Temporary redirect.

(Used mostly in browser-based apps)

==========
4xx — Client Error Codes
==========

400 Bad Request

Client sent invalid input.

Examples:
✔ Invalid JSON
✔ Validation failure
✔ Missing fields

401 Unauthorized

Authentication required or invalid token.

Meaning:
User not authenticated.

403 Forbidden

User authenticated but not allowed.

Meaning:
Access denied.

404 Not Found

Requested resource does not exist.

Example:
History ID not found.

405 Method Not Allowed

Wrong HTTP method used.

Example:
Sending POST on GET-only endpoint.

409 Conflict

Conflict with current state.

Example:
Duplicate resource creation.

422 Unprocessable Entity

Request is syntactically correct but semantically invalid.
(Common in validation-heavy APIs)

==========
5xx — Server Error Codes
==========

500 Internal Server Error

Unexpected server failure.

Used for:
✔ Unhandled exceptions
✔ Unknown runtime errors

502 Bad Gateway

Upstream service failure.

Example:
Microservice dependency failure.

503 Service Unavailable

Service temporarily unavailable.

Example:
Server overload
Maintenance mode

504 Gateway Timeout

Upstream service timed out.

==========
Most Important Codes for Interviews
==========

Must Remember:

200 → Success
201 → Created
204 → No Content
400 → Bad Request
401 → Unauthorized
403 → Forbidden
404 → Not Found
409 → Conflict
500 → Internal Server Error
503 → Service Unavailable

==========
Difference Between Commonly Confused Codes
==========

401 vs 403
---
401 → Not authenticated
403 → Authenticated but not authorized

400 vs 422
---
400 → Malformed request
422 → Validation/business rule failure

404 vs 403
---
404 → Resource doesn't exist
403 → Exists but no access

==========
How Spring Sets Response Codes
==========

Using @ResponseStatus:

@ResponseStatus(HttpStatus.NOT_FOUND)

Using ResponseEntity:

return ResponseEntity.status(404).body(...);

Using ExceptionHandler:

@ExceptionHandler(...)
@ResponseStatus(HttpStatus.BAD_REQUEST)

==========
Interview-Ready Summary
==========
HTTP response codes are standardized 3-digit numbers that indicate the outcome of an HTTP request. They are grouped into 1xx informational, 2xx success, 3xx redirection, 4xx client errors, and 5xx server errors. Proper use of status codes improves API clarity, debugging, and contract correctness in distributed systems.

==========
END
==========

Service Boundry

Service Boundary – Brief Summary

A **service boundary** defines what responsibilities, data, and business capability belong inside a microservice.

Key Principles

- **Business Capability Based**
  - Each service represents one clear business function.

- **Owns Its Data**
  - A service has its own database.
  - No direct DB access from other services.

- **High Cohesion**
  - All functionalities inside the service are closely related.

- **Low Coupling**
  - Avoid excessive synchronous calls between services.

- **Independent Deployment**
  - Services should be deployable without coordinating with others.

- **Change Isolation**
  - Features that change together → same service.
  - Features that evolve independently → separate services.

- **Transaction Boundary**
  - Avoid frequent distributed transactions.
  - Prefer eventual consistency (Saga pattern).

One-Line Definition

> A service boundary defines the scope of responsibility and data ownership of a microservice to ensure high cohesion, low coupling, and independent scalability.

Custom Aspect

Java

```java
import java.lang.annotation.*;

@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
@Documented
public @interface CheckRole {
    String value();   // role required
}
```

Use Annotation on Method

Java

```java
import org.springframework.stereotype.Service;

@Service
public class OrderService {

    @CheckRole("ADMIN")
    public void createOrder() {
        System.out.println("Order Created");
    }
}
```

JDBC

2. JDBC Repository (Using Plain JDBC)

Java

```java
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class UserRepository {

    private static final String URL = "jdbc:mysql://localhost:3306/testdb";
    private static final String USER = "root";
    private static final String PASSWORD = "password";

    public List<UserDTO> getAllUsers() {

        List<UserDTO> users = new ArrayList<>();
        String query = "SELECT id, name, email, status FROM users";

        try (
            Connection connection = DriverManager.getConnection(URL, USER, PASSWORD);
            PreparedStatement ps = connection.prepareStatement(query);
            ResultSet rs = ps.executeQuery()
        ) {

            while (rs.next()) {
                UserDTO user = new UserDTO(
                        rs.getLong("id"),
                        rs.getString("name"),
                        rs.getString("email"),
                        rs.getString("status")
                );
                users.add(user);
            }

        } catch (SQLException e) {
            throw new RuntimeException("Error fetching users", e);
        }

        return users;
    }
}
```

Dynamic rules engine

Dynamic Rule Engine (Teammate+) – Complete Interview Explanation

1. Overview

In Teammate+, a **dynamic rule engine** is implemented to allow business logic to be configured without modifying application code. Instead of hardcoding conditions in Java services, rules are created through an **admin UI page**, stored in the **database**, and evaluated dynamically at runtime.

This design enables business teams or admuinistrators to change system behavior without redeploying the application.

Typical use cases:

- Assessment validation rules
- Risk classification based on score
- Conditional workflow triggers
- Approval requirements
- Dynamic scoring logic
- Conditional notifications
- Role-based workflow actions

---

2. Problem With Hardcoded Business Logic

Without a rule engine, business logic is embedded directly in code.

Example:

```java
if(score < 40){
    assessment.setRiskLevel("HIGH");
}
```

Problems with this approach:

- Requires code changes for every rule update
- Requires deployment
- Business users cannot manage logic
- Difficult to scale rule complexity

A dynamic rule engine externalizes this logic into the database.

---

3. Core Idea of Dynamic Rule Engine

Rules are stored as **data instead of code**.

Example rule stored in DB:

rule_name	condition_expression	action_expression
HIGH_RISK_RULE	score < 40	setRiskLevel("HIGH")

At runtime:

1. Rules are loaded from the database
2. Conditions are evaluated dynamically
3. If the condition is true, the action is executed

---

4. High Level Architecture

Admin UI
↓
Rule Management API
↓
Database (Rules Table)
↓
Rule Engine Service
↓
Load Rules
↓
Evaluate Conditions
↓
Execute Actions
↓
Return Modified Entity

---

5. Rule Creation Flow

Step 1 — Admin Creates Rule From UI

Field	Value
Rule Name	High Risk Assessment
Entity Type	Assessment
Condition	score < 40
Action	setRiskLevel("HIGH")
Priority	10

| Active | true |

Step 2 — Rule Stored In Database

Example table:

```sql
CREATE TABLE rules (
    id BIGINT PRIMARY KEY,
    rule_name VARCHAR(255),
    entity_type VARCHAR(100),
    condition_expression TEXT,
    action_expression TEXT,
    priority INT,
    is_active BOOLEAN,
    created_at TIMESTAMP,
    updated_at TIMESTAMP
);
```

Example record:

id	rule_name	entity_type	condition_expression	action_expression	priority	is_active
1	HIGH_RISK_RULE	ASSESSMENT	score < 40	setRiskLevel("HIGH")	10	true

---

6. Rule Execution Flow

When an event occurs (example: assessment submission):

Assessment Submitted
↓
Rule Engine Service
↓
Fetch Active Rules From Database
↓
Filter Rules By Entity Type
↓
Sort Rules By Priority
↓
Evaluate Condition
↓
Execute Action
↓

Return Updated Object

---

7. Core Components

Rule Entity

```java
public class Rule {

    private Long id;
    private String ruleName;
    private String entityType;
    private String conditionExpression;
    private String actionExpression;
    private Integer priority;
    private Boolean active;
}
```

Rule Repository

```java
@Repository
public interface RuleRepository extends JpaRepository<Rule, Long> {

    List<Rule> findByEntityTypeAndActiveTrueOrderByPriorityDesc(String entityType);
}
```

Ensures:

- Only active rules are loaded
- Rules executed in priority order

Rule Engine Service

```java
@Service
public class RuleEngineService {

    @Autowired
    private RuleRepository ruleRepository;
```

```
    public void evaluateRules(Assessment assessment){

        List<Rule> rules = ruleRepository
                .findByEntityTypeAndActiveTrueOrderByPriorityDesc("ASSESSMENT");

        for(Rule rule : rules){

            if(evaluateCondition(rule.getConditionExpression(), assessment)){
                executeAction(rule.getActionExpression(), assessment);
            }
        }
    }
}
```

---

8. Condition Evaluation

Conditions stored as expressions.

Examples:

```
score < 40
department == 'Finance'
riskScore > 80
assessmentType == 'INTERNAL'
```

Evaluated using **Spring Expression Language (SpEL)**.

```java
private boolean evaluateCondition(String expression, Assessment assessment){

    ExpressionParser parser = new SpelExpressionParser();

    EvaluationContext context = new StandardEvaluationContext(assessment);

    Boolean result = parser.parseExpression(expression).getValue(context, Boolean.class);

    return Boolean.TRUE.equals(result);
}
```

---

9. Action Execution

Actions executed when condition is satisfied.

Examples:

```
setRiskLevel("HIGH")
setApprovalRequired(true)
assignReviewer("RISK_TEAM")
sendNotification("ADMIN")
```

Example implementation:

```java
private void executeAction(String action, Assessment assessment){

    if(action.contains("setRiskLevel")){
        assessment.setRiskLevel("HIGH");
    }

    if(action.contains("setApprovalRequired")){
        assessment.setApprovalRequired(true);
    }
}
```

In advanced systems actions use **strategy pattern or handler classes**.

---

10. Rule Caching

Fetching rules from the database every request is inefficient.

Rules are cached in memory.

Application Startup
↓
Load Rules From Database
↓
Store In Memory Cache
↓
Runtime Rule Evaluation

Example cache:

```
Map<String, List<Rule>> ruleCache
```

Example:

```
ASSESSMENT -> [Rule1, Rule2, Rule3]
SURVEY -> [Rule4, Rule5]
```

Cache refresh strategies:

- Scheduled refresh
- Event-based refresh
- Manual refresh

---

11. Rule Priority Handling

Multiple rules may apply.

Rule	Condition	Action
HIGH_RISK_RULE	score < 40	setRiskLevel("HIGH")
CRITICAL_RISK_RULE	score < 30	setRiskLevel("CRITICAL")

Priority:

Rule	Priority
CRITICAL_RISK_RULE	20
HIGH_RISK_RULE	10

Higher priority executes first.

---

12. Rule Versioning

Track rule updates.

```sql
CREATE TABLE rule_versions (
    id BIGINT PRIMARY KEY,
    rule_id BIGINT,
    version INT,
```

```
    condition_expression TEXT,
    action_expression TEXT,
    created_at TIMESTAMP
);
```

Benefits:

- Audit history
- Rollback capability
- Track rule evolution

---

13. Rule Execution Logging

```sql
CREATE TABLE rule_execution_log (
    id BIGINT PRIMARY KEY,
    rule_id BIGINT,
    entity_id BIGINT,
    result VARCHAR(50),
    executed_at TIMESTAMP
);
```

Example log:

rule_id	entity_id	result
1	4521	SUCCESS

---

14. Example End-to-End Scenario

Input:

```
score = 35
department = Finance
```

Rules:

Rule	Condition	Action
Rule1	score < 40	setRiskLevel("HIGH")

| Rule2 | department == 'Finance' | setApprovalRequired(true) |

Execution:

```
Rule1 triggered → riskLevel = HIGH
Rule2 triggered → approvalRequired = true
```

Final result:

```
riskLevel = HIGH
approvalRequired = true
```

---

15. Advantages

- Dynamic business logic
- No deployment required
- Business-friendly configuration
- Centralized rule management
- Extensible architecture

---

16. Limitations

- Complex debugging
- Rule conflicts
- Performance concerns for large rule sets

---

17. Interview Summary Answer

In Teammate+, a dynamic rule engine externalizes business logic from application code. Administrators create rules via a UI where each rule contains a condition and action. These rules are stored in the database and evaluated dynamically at runtime using Spring Expression Language. The rule engine loads active rules, evaluates conditions, executes actions, and applies them to entities. Rules are prioritized and cached in memory to improve performance. This allows business logic to change without modifying code or redeploying the application.

Authorisation

Authorisation

Previous rbac

==========
PREVIOUS AUTHORIZATION SETUP
(Static / Hard-coded Roles)
==========

1) PREDEFINED ROLES IN THE SYSTEM

The application had predefined roles already configured in the backend code.
These roles were fixed and could NOT be modified by the admin.

Example predefined roles:

ADMIN
AUDITOR
REVIEWER
OBSERVER

Internally Spring Security stores roles with prefix ROLE_ :

ROLE_ADMIN
ROLE_AUDITOR
ROLE_REVIEWER
ROLE_OBSERVER

---

2) ADMIN CREATES USER AND ASSIGNS ROLE

Admin could create a new user and assign one of the predefined roles.

Example:

User	Assigned Role
Tejas	ADMIN
Rahul	AUDITOR
Priya	REVIEWER

Example DB record:

username : rahul
role : ROLE_AUDITOR

---

3) PROTECTING APIs USING @PreAuthorize

To restrict access to APIs we used Spring Security annotation @PreAuthorize.

Example:

```
@PreAuthorize("hasRole('ADMIN')")
public void deleteAudit(Long id){
    auditRepository.deleteById(id);
}
```

Meaning:

Only users with ADMIN role can execute this method.

---

4) HOW @PreAuthorize CHECKS ROLE

When request comes:

```
Client Request
    ↓
JWT Token Validated
    ↓
Roles extracted from token
    ↓
Authentication object created
    ↓
Stored inside SecurityContext
```

Example SecurityContext:

```
User : rahul
Authorities : [ROLE_AUDITOR]
```

Now Spring evaluates:

```
@PreAuthorize("hasRole('ADMIN')")
```

Spring internally converts this to:

ROLE_ADMIN

Then it checks:

Does ROLE_ADMIN exist in authentication authorities?

---

5) FINAL AUTHORIZATION DECISION

CASE 1 : ROLE PRESENT

Authorities : [ROLE_ADMIN]
Required : ROLE_ADMIN

Result → Access Granted
Method executes

CASE 2 : ROLE NOT PRESENT

Authorities : [ROLE_AUDITOR]
Required : ROLE_ADMIN

Result → AccessDeniedException
HTTP 403 Forbidden
Method execution blocked

SHORT SUMMARY

- Roles were predefined in the system
- Admin could only assign roles to users
- APIs were protected using @PreAuthorize
- Spring fetched roles from SecurityContext
- If role matched → method executed
- If role missing → access denied

==========
Circuit breaker config
==========

==========

RESILIENCE4J CIRCUIT BREAKER - SETUP NOTES
==========

PURPOSE

Circuit Breaker protects an application from repeatedly calling a failing service.
If a service fails frequently, the circuit opens and prevents further calls,
redirecting requests to a fallback method.

Example:
Primary Service : Redis
Fallback Service: Database

---
1. ADD DEPENDENCY (MAVEN)

```
<dependency>
   <groupId>org.springframework.cloud</groupId>
   <artifactId>spring-cloud-starter-circuitbreaker-resilience4j</artifactId>
</dependency>
```

---
2. ENABLE ACTUATOR (FOR MONITORING)

```
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

---
3. CONTROLLER IMPLEMENTATION

```
package in.ashokit.rest;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

import io.github.resilience4j.circuitbreaker.annotation.CircuitBreaker;

@RestController
public class DemoRestController {

   @GetMapping("/data")
```

```
    @CircuitBreaker(
        name = "ashokit",
        fallbackMethod = "getDataFromDB"
    )
    public String getDataFromRedis() {

        System.out.println("Calling Redis Service...");

        // Simulating failure
        int i = 10 / 0;

        return "Data fetched from Redis";
    }


    // FALLBACK METHOD
    // Must have same return type
    // Must accept Throwable parameter

    public String getDataFromDB(Throwable t) {

        System.out.println("Fallback triggered: calling DB");

        return "Data fetched from Database";
    }
}


-------------------------------------------------
4. APPLICATION.YML CONFIGURATION
-------------------------------------------------

server:
  port: 8081

spring:
  application:
    name: resilience4j-demo


management:
  endpoints:
    web:
      exposure:
        include: '*'

  endpoint:
    health:
```

```
    show-details: always

  health:
    circuitbreakers:
      enabled: true


resilience4j:
  circuitbreaker:
    configs:
      default:

        registerHealthIndicator: true

        slidingWindowSize: 10
        minimumNumberOfCalls: 5

        permittedNumberOfCallsInHalfOpenState: 3

        automaticTransitionFromOpenToHalfOpenEnabled: true

        waitDurationInOpenState: 30s

        failureRateThreshold: 50

        eventConsumerBufferSize: 10
```

---

5. IMPORTANT CONFIGURATION PARAMETERS

---

slidingWindowSize

Number of requests used to calculate failure rate.

Example:
slidingWindowSize = 10
Circuit breaker tracks last 10 calls.

minimumNumberOfCalls

Minimum calls required before calculating failure rate.

Example:
minimumNumberOfCalls = 5

failureRateThreshold

Percentage of failures allowed before opening circuit.

Example:
failureRateThreshold = 50

If more than 50% requests fail → circuit opens.

waitDurationInOpenState

Time circuit stays OPEN before testing service again.

Example:
waitDurationInOpenState = 30s

permittedNumberOfCallsInHalfOpenState

Number of test calls allowed when circuit is HALF_OPEN.

Example:
3 test requests allowed.

---
6. CIRCUIT BREAKER STATES
---

1. CLOSED

Normal operation.
Requests go to primary service.

2. OPEN

Service failure detected.
Primary service calls are blocked.
Fallback method is executed.

3. HALF_OPEN

After wait duration expires.
Limited calls are allowed to test service.

If successful → circuit closes.
If failure → circuit opens again.

---
7. REQUEST FLOW

CLIENT
  |
  v
CIRCUIT BREAKER
  |
  v
PRIMARY SERVICE (REDIS)
  |
  | success
  v
RESPONSE TO CLIENT

If failure occurs:

CLIENT
  |
  v
CIRCUIT BREAKER
  |
  v
PRIMARY SERVICE FAILS
  |
  v
FALLBACK METHOD
  |
  v
DATABASE RESPONSE

---
8. MONITORING CIRCUIT BREAKER

Check circuit breaker status using actuator:

http://localhost:8081/actuator/health

or

http://localhost:8081/actuator/circuitbreakers

---
9. KEY INTERVIEW POINT

Circuit Breaker prevents cascading failures in microservices by temporarily stopping calls to a failing service and redirecting requests to a fallback mechanism.

States of circuit breaker
==========

WHICH LOGIC RUNS IN WHICH STATE

STATE PRIMARY LOGIC (REDIS) FALLBACK

CLOSED YES Only if exception occurs

OPEN NO Always executed

HALF_OPEN Limited requests If those requests fail

==========

RBAC AUTHORIZATION FLOW – PROJECT NOTES

(Multi-Tenant + Assessment Based Access)

SYSTEM IDENTIFIERS

tenantId → identifies client organization
userId (UUID) → identifies user
assessmentId → identifies audit/assessment context

Authorization decision is based on:

tenantId + userId + assessmentId

==========

==========

A) ADMIN OPERATIONS

---

1. ADMIN CREATES USER AND ASSIGNS ROLE

Admin creates a user from the UI and assigns role per assessment.

Example UI action:

User Email: tejas@gmail.com
Tenant: ABC Bank
Assessment: A101
Role: AUDITOR

DATABASE TABLES USED

TABLE: users

user_id (UUID)
tenant_id
email
name
status

Example:

user_id = U123
tenant_id = T100
email = tejas@gmail.com

TABLE: assessments

assessment_id
tenant_id
assessment_name

Example:

assessment_id = A101

TABLE: user_assessment_roles

Stores role of a user for a specific assessment.

id
tenant_id
user_id
assessment_id
role_id
assigned_at

Example record:

tenant_id T100
user_id U123
assessment_id A101
role_id AUDITOR

Meaning:

User U123 is AUDITOR in assessment A101

---

2. ROLE → PERMISSION MAPPING

Roles and permissions are stored separately.

TABLE: roles

role_id
role_name

Example:

ADMIN
AUDITOR
REVIEWER

TABLE: permissions

permission_id
permission_key

Example:

AUDIT_CREATE
AUDIT_EDIT
WORKPAPER_UPLOAD
REPORT_EXPORT

TABLE: role_permissions

role_id
permission_id

Example records:

AUDITOR → AUDIT_CREATE
AUDITOR → AUDIT_EDIT
AUDITOR → WORKPAPER_UPLOAD

REVIEWER → AUDIT_REVIEW
REVIEWER → WORKPAPER_APPROVE

ADMIN → all permissions

==========

==========

B) USER REQUEST FLOW

---

1. HOW REQUEST URL LOOKS

Example request:

POST /api/v1/assessments/A101/workpapers

Headers:

Authorization: Bearer JWT_TOKEN

From URL we extract:

assessmentId = A101

---

2. JWT PAYLOAD CONTENT

JWT contains identity information only.

Example payload:

```
{
   "tenantId": "T100",
   "userId": "U123",
   "email": "tejas@gmail.com", or we can send uuid
   "exp": 1712345678
}
```

Important fields used:

tenantId
userId

These values are extracted during authentication.

---

3. CUSTOM AUTHORIZATION ANNOTATION

APIs are secured using a custom annotation.

Example:

@RequirePermission("WORKPAPER_UPLOAD")

API example:

```
@RequirePermission("WORKPAPER_UPLOAD")
@PostMapping("/assessments/{assessmentId}/workpapers")
public ResponseEntity uploadWorkpaper() {
    ...
}
```

Meaning:

Only users having permission WORKPAPER_UPLOAD
for that assessment can execute this API.

---

4. AOP AUTHORIZATION ASPECT

The aspect intercepts the API and checks permission.

```
@Aspect
@Component
public class AuthorizationAspect {

    @Autowired
    private RedisService redisService;

    @Autowired
    private RoleService roleService;

    @Before("@annotation(requirePermission)")
    public void checkPermission(RequirePermission requirePermission) {

        String requiredPermission = requirePermission.value();

        Authentication auth =
            SecurityContextHolder.getContext().getAuthentication();
```

```
        String userId = auth.getName();

        HttpServletRequest request =
            ((ServletRequestAttributes) RequestContextHolder
                .getRequestAttributes()).getRequest();

        String tenantId = request.getHeader("tenantId");
        String assessmentId = request.getParameter("assessmentId");

        String role = redisService.getUserRole(tenantId, userId, assessmentId);

        Set<String> permissions = redisService.getPermissionsForRole(role);

        if(!permissions.contains(requiredPermission)){
            throw new AccessDeniedException("Permission denied");
        }
    }
}
```

---

5. REDIS CONFIGURATION AND METHODS

Redis is used as a caching layer for RBAC.

Main Redis Keys:

User role per assessment:

role:user:{tenantId}:{userId}:{assessmentId}

Example:

role:user:T100:U123:A101 → AUDITOR

Role permissions:

perm:role:{roleName}

Example:

perm:role:AUDITOR

Value stored as SET:

AUDIT_CREATE
AUDIT_EDIT
WORKPAPER_UPLOAD

Redis Service Example:

```
public String getUserRole(String tenantId,String userId,String assessmentId){

    String key = "role:user:" + tenantId + ":" + userId + ":" + assessmentId;

    return redisTemplate.opsForValue().get(key);
}


public Set<String> getPermissionsForRole(String role){

    String key = "perm:role:" + role;

    return redisTemplate.opsForSet().members(key);
}
```

---

6. CACHE INSERTION WITH TTL

When cache miss occurs we fetch from DB and store in Redis.

```
public void cacheUserRole(String tenantId,String userId,String assessmentId,String role){

    String key = "role:user:" + tenantId + ":" + userId + ":" + assessmentId;

    redisTemplate.opsForValue().set(key, role, 15, TimeUnit.MINUTES);
}
```

Role permissions cache:

```
public void cacheRolePermissions(String role, Set<String> permissions){

    String key = "perm:role:" + role;

    redisTemplate.opsForSet().add(key, permissions.toArray(new String[0]));

    redisTemplate.expire(key, 30, TimeUnit.MINUTES);
```

}

---

7. GRANULAR CACHE EVICTION

When admin updates role or permission we delete only
specific keys.

Example role change:

redisTemplate.delete("perm:role:AUDITOR");

Example user role update:

redisTemplate.delete("role:user:T100:U123:A101");

Next request will repopulate cache.

---

8. MEMORY BASED EVICTION POLICY

Redis configuration:

maxmemory 4gb
maxmemory-policy allkeys-lfu

Meaning:

Redis removes least frequently used keys
when memory is full.

---

9. IMPORTANT THINGS TO FOCUS

• Use userId(UUID) instead of email for Redis keys
• Avoid storing roles inside JWT
• Always invalidate cache after role updates
• Use Redis SET for permission lookup
• Use cache-aside pattern for loading RBAC data

• Never use global FLUSHALL in production
• Ensure Redis TTL prevents stale permissions

==========

==========

FINAL AUTHORIZATION FLOW

User Request
↓
JWT validated
↓
Extract tenantId + userId
↓
Extract assessmentId from API
↓
Redis lookup → role:user:{tenantId}:{userId}:{assessmentId}
↓
Role obtained
↓
Redis lookup → perm:role:{roleName}
↓
Permission list obtained
↓
Aspect checks required permission
↓
Allow request OR throw AccessDeniedException

==========

==========

END

Hackthon

Hackathon Project: AI-Driven Risk Auditing(Code Games 2024 – 1st Place)

Summary:

Built an AI-powered enterprise risk intelligence platform to assist auditors in identifying similar historical risks and automatically generating mitigation controls. The system leveraged semantic vector search and large language models to improve risk discovery beyond traditional keyword-based search systems.

Core Technologies Used:
- Backend: Java, Spring Boot (REST APIs)
- AI/LLM: OpenAI GPT-3.5 Turbo
- Embeddings: OpenAI Text Embeddings
- Vector Database: Pinecone (vector embeddings)
- Frontend: React
- Architecture: RAG-style semantic retrieval + LLM generation
RAG -> retrieval augmented generation
Technical Implementation:

1. Semantic Risk Representation
Risk descriptions were converted into high-dimensional vector embeddings using OpenAI embedding models.
These embeddings capture semantic meaning, allowing detection of risks that are conceptually similar even when the wording differs.

2. Vector Storage and Retrieval
Embeddings were stored in Pinecone vector database.
For each new risk submitted by an auditor:
- The risk text was converted to an embedding
- A similarity search (cosine similarity) was executed in Pinecone
- The system retrieved the top K most semantically similar historical risks

3. Semantic Risk Search Pipeline
New Risk Input
↓
Embedding Generation (OpenAI API)
↓
Vector Query in Pinecone
↓
Top-K Similar Historical Risks
↓
Context sent to LLM

4. AI Risk and Control Generation
The retrieved risks were used as context for GPT-3.5 Turbo to generate:
- New potential lookalike risks
- Recommended mitigation controls
- Compliance suggestions

Prompt structure included:
- current risk
- similar historical risks
- control examples

5. Backend API Flow

POST /risk/analyze

Steps:
1. Receive risk description
2. Generate embedding via OpenAI API
3. Query Pinecone for similar vectors
4. Send retrieved risks to GPT-3.5
5. Return generated risks and mitigation controls

6. Key Technical Concepts Implemented

Vector Embeddings
Transforms text into numeric vectors that represent semantic meaning.

Approximate Nearest Neighbor Search
Used by Pinecone to efficiently find similar vectors among large datasets.

Semantic Retrieval
Enables contextual similarity search rather than keyword matching.

LLM-Based Risk Generation
GPT-3.5 generated additional risks and suggested mitigation strategies using retrieved context.

7. System Outcome

The platform enabled:
- Semantic search across historical audit risks
- AI-generated risk discovery
- Automated control recommendations

This significantly improved the efficiency of risk analysis and audit preparation, which led to the project winning **1st place in the ESG category at the Code Games 2024 hackathon**.